

1) CREATE DATABASE mystartensql → creates DB

2) USE mystartensql

⊗ SHOW CREATE TABLE table-name

3) CREATE TABLE users (

id INT AUTO INCREMENT PRIMARY KEY } constraints
name → Type

name VARCHAR(150) NOT NULL,

age INT, gender ENUM('MALE', 'FEMALE', 'OTHER');

email VARCHAR(150) UNIQUE NOT NULL,

dob DATE,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;

4) SELECT * FROM users; → Select all the cols

SELECT amount * 100
FROM USER

5) SELECT name, email FROM users; → Select specific cols

6) RENAME TABLE users to customers; → Rename Table

7) ALTER TABLE users ADD COLUMN is_active BOOLEAN DEFAULT TRUE;

ALTER TABLE users RENAME COLUMN is_active TO status;

8) ALTER TABLE users DROP COLUMN is_active

to change datatype

9) ALTER TABLE users MODIFY COLUMN email VARCHAR(200);

10) ALTER TABLE users email VARCHAR(200) First → moves the
type required email first position

11) ALTER TABLE users dob DATE AFTER name; → move col after
name

INSERTION

12) INSERT INTO users VALUES

(1, 19, 'MALE', 'mondal@gmail.com', '2006-04-21'); → Raw method
(Not Recommended)

13) INSERT INTO users (name, email, age, gender, dob) VALUES
('nik', 'mn@gmail.com', 19, MALE, 2006-04-21); → Data Insert

For multiple Insertion

(- - -),

(- - -);

Filteration

- 14) SELECT * FROM users WHERE gender = 'MALE'; → Similarly =, !=, <, >, >=, <=
- 15) SELECT * FROM users WHERE DOB IS NULL. Similarly IS NOT NULL
- 16) SELECT * FROM users WHERE DOB BETWEEN _____ AND _____
- 17) SELECT * FROM users WHERE gender IN ('MALE', 'OTHER');

PATTERN / LIKE

LIKE %V%V%

LIKE 'E-AM'

- 18) SELECT * FROM users WHERE name LIKE 'A%'; → name starts with A
- 19) ' %A' → name ends with 'A'
- 20) ' %li%' → name contains 'li'

AND/OR

- 21) SELECT * FROM users WHERE name = 'Rik' AND age = 19;

OR

- 23) SELECT * FROM users ORDER BY DOB, ASC → selection on ascending descending order.
- 24) ~~DESC~~ DESC

LIMIT

- 25) SELECT * FROM users ~~with~~ LIMIT 5; → Select only first 5 rows
- 26) SELECT * FROM users LIMIT 5, 10; → Select 10 rows starting from 5
- 27) SELECT * FROM users LIMIT 10, OFFSET 5; →

UPDATE

- 28) ~~SELECT~~ UPDATE users SET email = 'moik@gmail.com', DOB = '2006-04-21' WHERE id = 5;

DELETE

- 29) DELETE FROM users WHERE id = 3;

UNIQUE

- 30) CREATE TABLE users (
id INT PRIMARY KEY,
email VARCHAR(50) UNIQUE
);

→ Adding constraint while creating table.

- 31) ALTER TABLE users

ADD CONSTRAINT unique_email UNIQUE(email) → Adding constraint later.
↓
constraint name

Naming constraint helps to do operation on that later.

NOT NULL & NULL
32> CREATE TABLE users (
email VARCHAR(200) NOT NULL
);

33> ALTER TABLE users MODIFY COLUMN email NULL

CHECK

34> ALTER TABLE users ADD CONSTRAINT chk_dob CHECK (dob > '2007-04-21')

DEFAULT

35> CREATE TABLE users (
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

36> ALTER TABLE users ALTER COLUMN is_active SET DEFAULT TRUE.

PRIMARY KEY

37> CREATE TABLE users (
id INT PRIMARY KEY);

38> ALTER TABLE users ADD PRIMARY KEY (id);

ALTER TABLE table_name
DROP PRIMARY KEY;

AUTO INCREMENT

39> CREATE TABLE users (
id INT AUTO INCREMENT);

40> ALTER TABLE users MODIFY COLUMN id AUTO INCREMENT;

55> ALTER TABLE users AUTO_INCREMENT = 1000; → Starts from here

Having → filter groups after grouping

SELECT COUNT(*) FROM STUDENT WHERE MARKS > 70 GROUP BY
Name HAVING COUNT(*) > 1;

FUNCTIONS

- 41) `SELECT COUNT(*) FROM users;`
- 42) `SELECT COUNT(*) FROM users WHERE gender = 'MALE';`
- MIN() & MAX()
- 43) `SELECT MIN(salary) AS min-salary, MAX(salary) AS max-salary FROM users;`
- 44) SUM()
- `SELECT SUM(salary) AS total FROM users;`
- 45) AVG()
- `SELECT AVG(salary) AS average FROM users;`

GROUP BY

- 46) `SELECT gender, AVG(salary) AS avg-salary FROM users GROUP BY gender;`
- Annotations:
 - `gender`: column names
 - `AVG(salary)`: output
 - `FROM users`: columns
 - `GROUP BY gender`: Aggregation

name	dept	marks
	CSE	9.0
	MC	9.0
	CSE	9.5
	BT	9.8
	MHE	9.2
	CSE	10

`SELECT DEPT, AVG(marks) AS avg-marks FROM users GROUP BY dept;`

- 47) `SELECT DEPT FROM students GROUP BY dept;`
- Return the departments available

LENGTH()

- 48) `SELECT name, LENGTH(name) AS name-length FROM students;`

LOWER() & UPPER()

`SELECT name, LOWER(name) AS lowercase-name FROM users;`

CONCAT()

`SELECT CONCAT(name, '<', email, '>') AS user-contact FROM users;`

NOW()

- 51) `SELECT NOW();`

SEPERATE Extraction of

`YEAR(), MONTH(), DAY()`

- 52) `SELECT YEAR(dob) FROM users;`

- 53) `SELECT DATEDIFF(CURDATE(), dob) AS days-lived FROM users;`

- 54) `SELECT TIMESTAMPDIFF(YEAR, CURDATE(), dob) AS age FROM users;`

Join

59) INNER JOIN → only matching Row

```
SELECT users.name, orders.name
```

```
FROM users
```

```
INNER JOIN addresses ON users.id = addresses.user_id
```

60) LEFT JOIN → All from left + matching from right others null

61) RIGHT JOIN

62) UNION

```
SELECT name FROM users
```

Table 1

```
UNION
```

Another table

```
SELECT name FROM admin_users
```

63) UNION ALL → keeps duplicate Name.

64) SELECT name, 'user' AS role FROM users

```
UNION
```

```
SELECT name, 'Admin' AS role FROM admin_users;
```

65) SELECT name FROM users

```
UNION
```

```
SELECT name FROM admin_users
```

```
GROUP BY name;
```

66)

1> CREATE VIEW high-salary AS SELECT id, name, salary
FROM users WHERE salary > 50000;

view always reflect to live data

2> DROP VIEW high-salary

1> SELECT id, name, salary FROM users WHERE (SALARY >
(SELECT AVG(salary) FROM users);

1> IN → multiple os

SELECT * FROM customers WHERE COUNTRY IN ('Germany',
'India', 'America');

2> NOT IN

3> SELECT * FROM student WHERE MARKS > ANY (

select marks from student WHERE COURSE
ID > 100);

ALL → ANY + AND

EXISTS, NOT EXISTS

1> SELECT name FROM student WHERE EXISTS (SELECT
courseid FROM courses WHERE courseid > 100);

Using AS for column Alias

SELECT Name as Sname FROM student;

SELECT Name, MARKS * 2 as new_marks FROM student;

WHILE JOINING

SELECT S.name, C.id FROM student as S JOIN courses
as C ON S.course_id = C.course_id.

1> SELECT amount FROM acc WHERE amount BETWEEN
10000 AND 20000 → similar where amount ≥ 10000
AND ≤ 20000

2> SELECT DISTINCT

3>

With

Temporary view

```
WITH TOP_COURSE AS (SELECT * FROM COURSE WHERE COURSE_ID > 100);  
SELECT * FROM TOP_COURSE;
```

CASE

```
SELECT NAME, CASE WHEN MARKS > 85 THEN 'EXCELLENT' WHEN  
CASE WHEN MARKS > 90 THEN 'OUTSTANDING'  
ELSE 'Good'  
END FROM students;
```

STRING OPERATION

```
SELECT CONCAT(firstname, ' ', lastname) AS Full-Name FROM --
```

UPPER()

SELECT LOWER(firstname) --

LENGTH()

SUBSTRING(string, start pos, end pos)

LOCATE(

REPLACE('Hello world', 'world', 'mysql') → hello mysql

TRIM(' Hello ') → Hello

```
SELECT Borrower FROM LOAN  
UNION  
SELECT Customer FROM depositon.
```

~~INTERSECT~~

SELECT COUNT(distinct name) from depositon.

SOME & ALL

> some (subquery) → greater than atleast one value

> All (subquery) → greater than every value

```
SELECT BRANCH_NAME FROM BRANCH WHERE AMOUNT > SOME(  
SELECT AMOUNT FROM BRANCH WHERE CITY = 'DGR');
```

SELECT MAX(cnt) FROM (

~~SELECT~~ SELECT COUNT(*) FROM DEPT GROUP BY DEPT_NAME)

AS T;

FOREIGN KEY

NOT RECOMMENDED

- 55) CREATE TABLE orders (
- id INT AUTO_INCREMENT PRIMARY KEY,
- FOREIGN KEY (order_id) REFERENCES users(id);
-);
- 56) ALTER TABLE orders
- ADD CONSTRAINT fk_order FOREIGN KEY (order_id) REFERENCES users(id);
- 57) ALTER TABLE orders
- DROP FOREIGN KEY fk_order;
- 58) CREATE TABLE orders (
- order_id INT,
- ADD CONSTRAINT ~~fk_order~~ FOREIGN KEY (order_id)
- REFERENCES users(id) ON DELETE CASCADE;
-)

DELETE OPTIONS

- CASCADE → on deleting parent deletes child
- SET NULL → on deleting parent set foreign key NULL
- RESTRICT → Prevent to delete the parent..

Foreign key always refers to unique or primary key attribute.